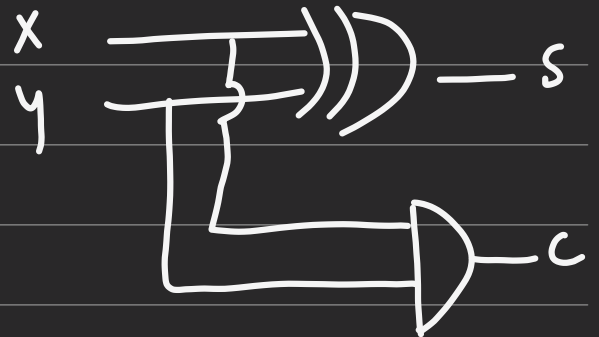


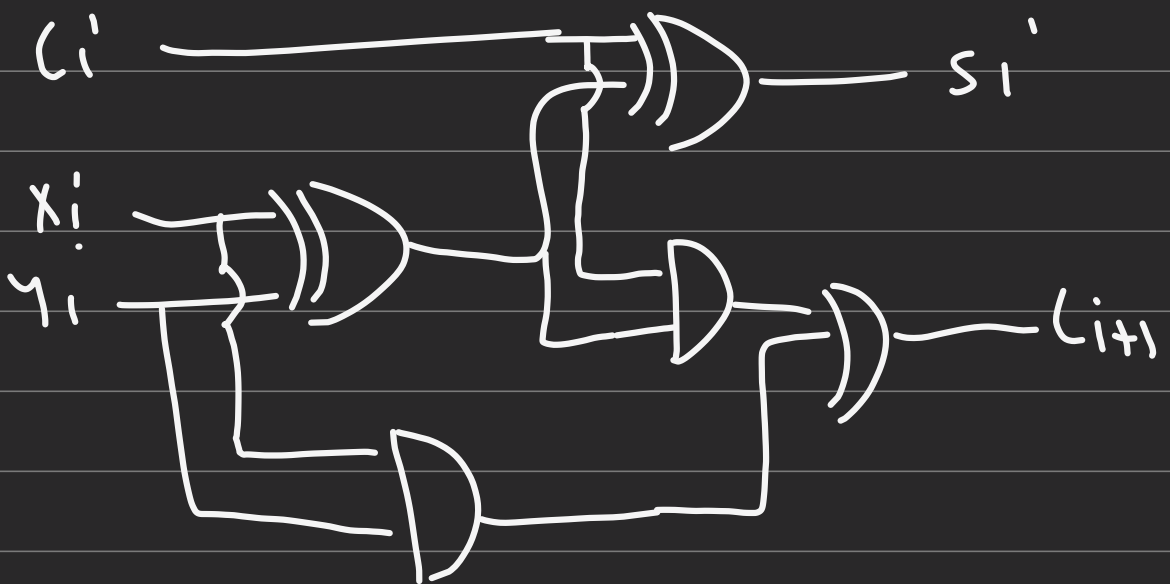
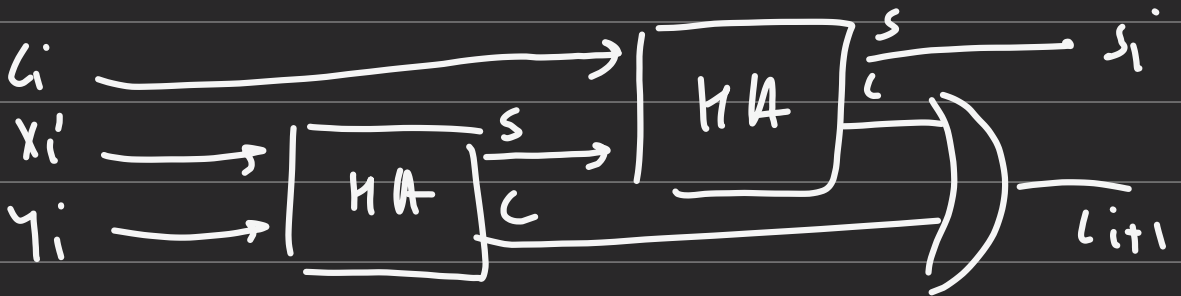
Adders

① Half Adder

$$S = x \oplus y$$
$$C = xy$$



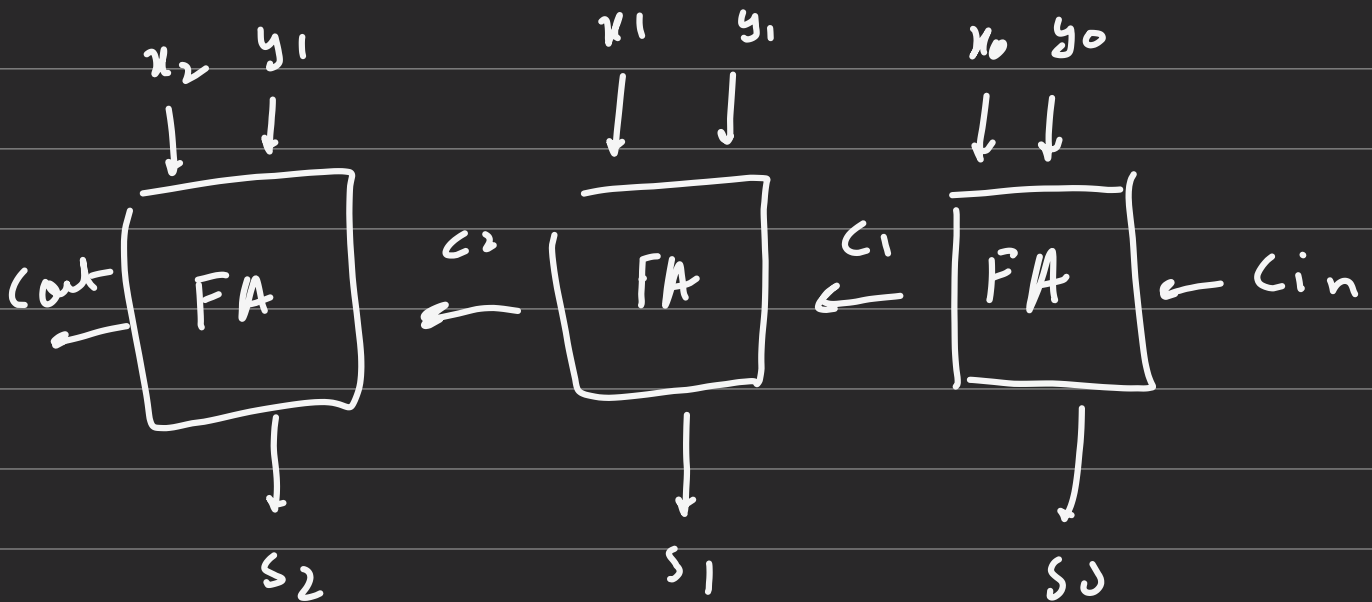
② Full Adder



$$S = x_i \oplus y_i \oplus C_i$$

$$C = (x_i \oplus y_i) C_i + x_i y_i$$

③ Ripple carry Adder



input $[3:0]$ X, Y
 ↗ least significant
 ↘ most significant

④ n bit RCA

(parameter $n = 4$)

(input —————)
 —————)
 will take this as default value

assign $[0] =$ —

$[n]$

generate ^{only when using int. inside gen loop.}
 genval k can't use k++
 for (k=0; k < n; k=k+1)
 begin : stage
 |
 end.
 end generate to give name of each instance

Subtractor

→ 2's Complement

① first 1's Complement
(flip all bits)

② Add 1

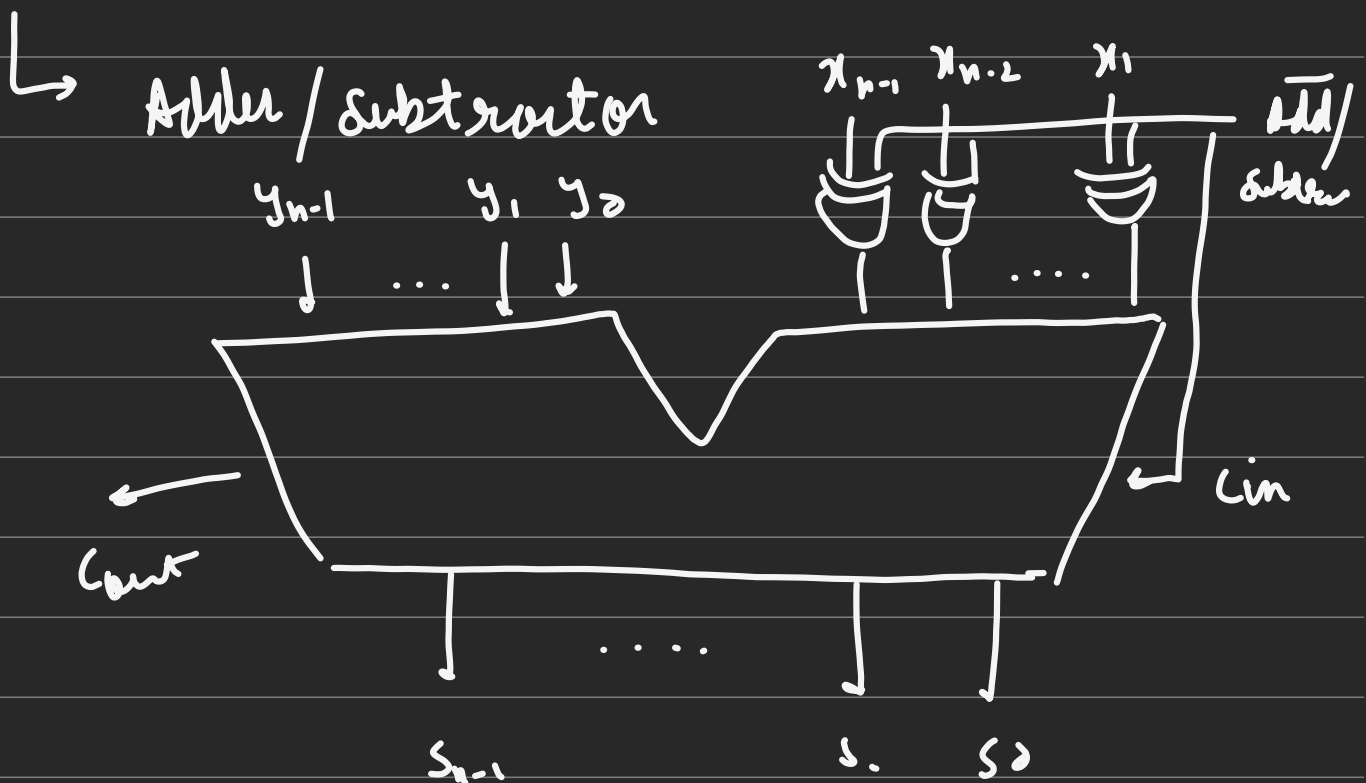
$$\begin{array}{r}
 \underline{10} \\
 1010 \\
 0101 \leftarrow \text{1's complement} \\
 +1 \\
 \hline
 0110 \leftarrow \text{2's complement} \\
 \hline
 \end{array}$$

↳ So subtraction will be adding 2's complement of it.

ic

$$\begin{array}{r}
 14 \\
 - 10 \\
 \hline
 4
 \end{array}
 \quad
 \begin{array}{r}
 1110 \\
 - 1010 \\
 \hline
 0100
 \end{array}
 \quad
 \begin{array}{r}
 1110 \\
 + 0110 \\
 \hline
 10100
 \end{array}$$

Arrows indicate the conversion of the decimal subtraction to binary and the addition of the 2's complement (10100) to the binary result (0100) to get the final result (10100).



Signed no. & Arithmetic overflow

4 bit no. (-8 to 7)

2's comp.

$ \begin{array}{r} +6 \quad 010 \\ +3 \quad 0011 \\ \hline +9 \quad 01001 \\ \hline \end{array} $ OR $ \begin{array}{r} -7 \\ +0110 \\ \hline \end{array} $ -1 & 1's comp.	$ \begin{array}{r} -6 \quad 1010 \\ +3 \quad 0011 \\ \hline -3 \quad 1101 \\ \hline \end{array} $
--	---

$ \begin{array}{r} +6 \quad 0110 \\ -3 \quad 1101 \\ \hline +3 \quad 10011 \\ \hline \end{array} $ +3	$ \begin{array}{r} -6 \quad 1010 \\ -3 \quad 1101 \\ \hline -9 \quad 10111 \\ \hline \end{array} $ +7
---	---

way to check overflow

if $\text{Carry in} \wedge \text{Carry out}$ of last bit is 1 then overflow.

M2 if MSB are 1 & output is 0 or sign bit.

→ assign $sum = x + y$

5 bit 4 bit

so + operator will automatically extend x to 5 bits & then add, so as per min length

Test bench

module _____ ();

- 1) Declare local reg & wire identifiers
- 2) Instantiate the module under test.
- 3) Specify a stopwatch to stop the simulation
- 4) Generate stimuli, using initial and always
- 5) Display the output response (text or graphics)

endmodule

i.e.

module _____ ();

parameter $n = 4$;

reg _____

wire _____

_____ ;

adder subtractor # ($n(n)$) out (_____)

initial

40 \$ finish

initial

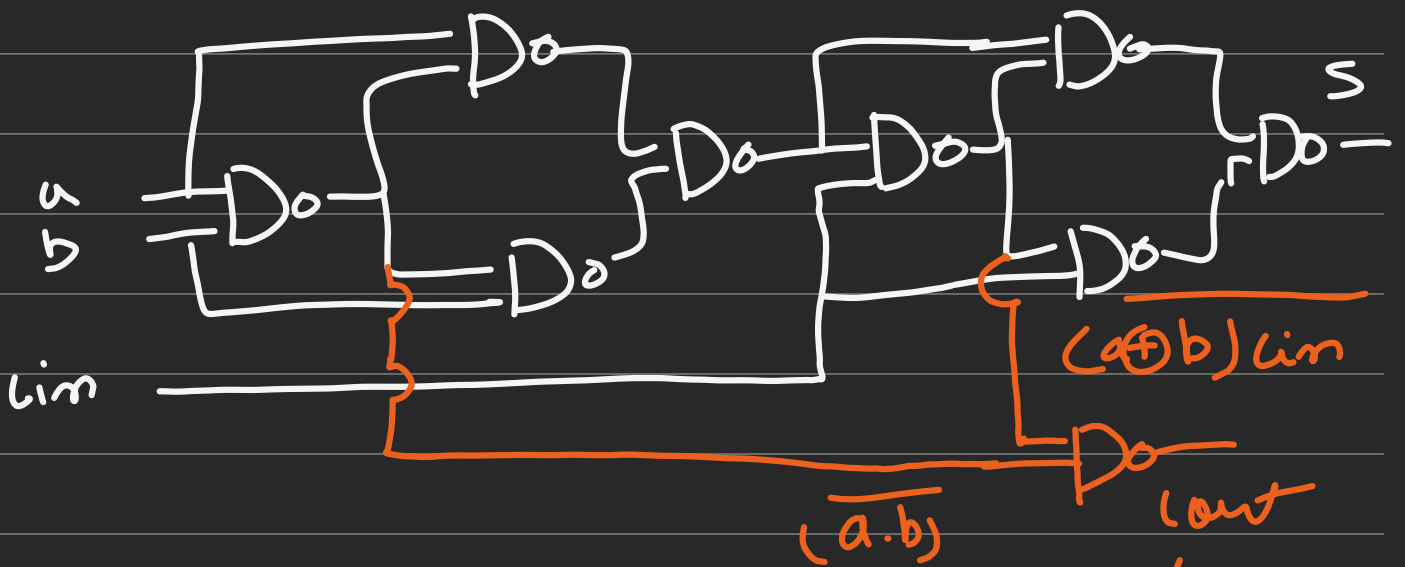
begin

10

10

end

Full Adder using NAND gates.



$$\overline{((a \cdot b) \cdot ((a \oplus b) cin))}$$

$$= (a \cdot b) + (a \oplus b) cin$$

